

Clinic AI Assistant — Day 1 PostMortem Deliverables

Purpose of this document: capture the real outcomes of Day 1 implementation, what was completed, what remains placeholder, and what architectural risks or improvements should be considered before moving forward.

Day 1 Achievements

- Local development environment configured (VS Code, Python, virtual environment)
- Project repository initialized and first commit created
- FastAPI backend skeleton established
- Environment variable loading verified via `.env`
- POST `/chat` endpoint implemented and validated via Swagger
- Basic request validation implemented using Pydantic

Current Architecture Status

Layer	Status	Notes
Documentation	Complete	Architecture docs, plan, task tracker exist
Repository	Complete	Git initialized, project structure stable
Backend runtime	Working	FastAPI running locally
Routing layer	Working	<code>/chat</code> endpoint implemented
Request validation	Working	Pydantic request model
Environment configuration	Working	<code>.env</code> loading verified
AI service layer	Placeholder	<code>ai_agent.py</code> prepared but unused
Frontend layer	Placeholder	frontend folder only
Deployment pipeline	Missing	No deployment configuration yet

Key Observations

- Code discipline rule established: every change must produce a measurable result
- Architecture kept intentionally minimal to avoid premature complexity
- Service layer prepared early to isolate AI logic from route layer
- Swagger docs proved useful for validating endpoints quickly

Technical Debt / Risks

- `ai_agent.py` not implemented yet
- No structured logging yet
- No error handling layer
- No request tracing or observability
- No testing framework yet

- No deployment configuration (Docker, hosting, CI)

Recommended Next Focus

- Implement AI service layer (OpenAI integration)
- Connect POST /chat endpoint to service layer
- Define response schema
- Add minimal logging
- Prepare knowledge retrieval design